Operator Overloading Python-ում

Ամբողջական տեղեկատու բոլոր գերբեռնելի օպերատորների և dunder method-ների մասին:

1. Մաթեմատիկական գործողություններ

Թվաբանական գործողությունները հատուկ dunder method-ների օգնությամբ կարող ենք սահմանել մեր կլասների համար:

Operator	Dunder method	Նկարագրություն	Օրինակ
`+`	`__add__(self, other)`	գումարում	`Vector(1,2) + Vector(3,4)`
`-`	`__sub__(self, other)`	հանում	`Vector(5,6) - Vector(1,2)`
`*`	`__mul__(self, other)`	բազմապատկում	`Vector(2,3) * 4`
`/`	`__truediv__(self, other)`	բաժանում	`Vector(4,6) / 2`
`//`	`__floordiv__(self, other)`	ամբողջ բաժանում	`Vector(7,9) // 2`
`%`	`__mod__(self, other)`	մնացորդ	`Vector(7,9) % 3`
`**`	`__pow__(self, other)`	աստիճան	`Vector(2,3) ** 2`
`(right) +`	`__radd__(self, other)`	աջ գումարում	`5 + Vector(1,2)`
`(right) -`	`__rsub__(self, other)`	աջ հանում	`5 - Vector(1,2)`
`(right) *`	`__rmul__(self, other)`	աջ բազմ.	`3 * Vector(2,3)`

2. Համեմատության օպերատորներ

Կլասի օբյեկտները կարելի է համեմատել, եթե սահմանենք համապատասխան dunder method-ները:

Operator	Dunder method	Նկարագրություն	Օրինակ
`==`	`__eq__(self, other)`	հավասար	`Temperature(20) == Temperature(20)`
`!=`	`__ne__(self, other)`	անհավասար	`Temperature(20) != Temperature(30)`
`<`	`__lt__(self, other)`	փոքր	`Temperature(10) < Temperature(20)`

>	`__gt__(self, other)`	մեծ	`Temperature(30) > Temperature(20)`
<=	`__le__(self, other)`	փոքր կամ հավասար	`Temperature(20) <= Temperature(20)`
>=	`__ge__(self, other)`	մեծ կամ հավասար	`Temperature(30) >= Temperature(20)`

3. In-place վերագրում (+=, -= ...)

In-place operator-ները փոփոխում են օբյեկտն ուղղակիորեն: Եթե չսահմանվեն, Python-ն օգտագործում է __add__-ի արդյունքը:

Operator	Dunder method	Նկարագրություն	Օրինակ
+=	`__iadd__(self, other)`	գումարում	`counter += 5`
-=	`__isub__(self, other)`	հանում	`counter -= 3`
***=**	`__imul__(self, other)`	բազմապատկում	`counter *= 2`
/=	`__itruediv__(self, other)`	բաժանում	`counter /= 2`
//=	`__ifloordiv__(self, other)`	ամբողջ բաժանում	`counter //= 3`
%=	`__imod__(self, other)`	մնացորդ	`counter %= 4`
****=**	`__ipow__(self, other)`	աստիճան	`counter **= 2`

4. Unary օպերատորներ

Unary operator-ները կիրառվում են մեկ օբյեկտի վրա (ոչ երկուսի):

Operator	Dunder method	Նկարագրություն	Օրինակ
-x	`__neg__(self)`	բացասական	`-money`
+x	`__pos__(self)`	դրական	`+money`
abs()	`__abs__(self)`	բացարձակ արժեք	`abs(money)`
~x	`__invert__(self)`	bit invert	`~flags`
round()	`__round__(self, n)`	կլորացում	`round(money, 2)`
int()	`__int__(self)`	ամբ. թիվ	`int(money)`

float()	`__float__(self)`	float	`float(money)`

5. Bit գործողություններ

Bit operator-ները հիմնականում օգտագործվում են ֆլագների (flags) կամ ցածր մակարդակի գործողությունների համար:

Operator	Dunder method	Նկարագրություն	Օրինակ
&	`__and__(self, other)`	AND	`perm1 & perm2`
|	`__or__(self, other)`	OR	`perm1 | perm2`
^	`__xor__(self, other)`	XOR	`perm1 ^ perm2`
<<	`__lshift__(self, n)`	ձախ shift	`flags << 1`
>>	`__rshift__(self, n)`	աջ shift	`flags >> 1`
&=	`__iand__(self, other)`	AND in-place	`perm &= mask`
|=	`__ior__(self, other)`	OR in-place	`perm |= Permission.WRITE`

6. Container / հաջորդականություն

Այս operator-ների օգնությամբ կարող ենք ստեղծել մեր սեփական list-ի կամ dict-ի նման կլասներ:

Operator	Dunder method	Նկարագրություն	Օրինակ
obj[i]	`__getitem__(self, key)`	կարդալ	`stack[0]`
obj[i]=x	`__setitem__(self, key, val)`	գրել	`stack[0] = 'x'`
del obj[i]	`__delitem__(self, key)`	ջնջել	`del stack[0]`
x in obj	`__contains__(self, item)`	պարունակել	`'a' in stack`

len()	`__len__(self)`	երկարություն	`len(stack)`
iter()	`__iter__(self)`	for loop	`for x in stack`
next()	`__next__(self)`	հաջ. element	`next(iterator)`

7. Հատուկ / repr դunder method-ներ

Այս method-ները կառավարում են, թե ինչպես Python-ն «տեսնում» ու «կատարում» է օբյեկտը:

Operator	Dunder method	Նկարագրություն	Օրինակ
str()	`__str__(self)`	print(obj)	`print(my_obj)`
repr()	`__repr__(self)`	debug	`repr(my_obj)`
bool()	`__bool__(self)`	True/False	`if my_obj:`
hash()	`__hash__(self)`	dict key	`d[my_obj] = 1`
obj()	`__call__(self, ...)`	ֆունկցիայի պես կանչ	`my_obj(5)`
obj.x	`__getattr__(self, name)`	attribute	`obj.unknown`
with	`__enter__ / __exit__`	context manager	`with obj as x:`